Course Title:

BIT302 Software Engineering

Course Contents

Unit-1: Introduction (2 Hrs.)
Professional Software Development, Software Engineering Ethics, Case Studies

Unit 2: Software Process Model (8 Hrs.)
Software Process Models, Process Activities, Coping with Change, Process Improvement, Agile Software Development

Unit 3: Requirements Engineering (3 Hrs.)
Functional and Non-Functional Requirements, Requirements Engineering Processes, Requirements Elicitation, Requirements Validation, Requirements Change

Unit 4: System Modeling (6 Hrs.)
Context Models, Interaction Models, Structural Models, Behavioral Models, Model Driven Architecture

Course Contents

Unit 5: Architectural Design (6 Hrs.)

Architectural Design Decisions, Architectural Views, Architectural Patterns, Application Architectures

Unit 6: Design and Implementation (5 Hrs.)

Object-Oriented Design Using the UML, Design Patterns, Implementation Issues, Open-Source Development

Unit 7: Software Testing (5 Hrs.)

Introduction to Software Testing, Development Testing, Test-Driven Development, Types of Testing

Unit 8: Project Management and Planning (5 Hrs.)

Introduction to Project Management, Management Activities, Introduction to Project Planning, Software Pricing, Plan-Driven Development, Project Scheduling, Agile Planning, Estimation Techniques, COCOMO Cost Modeling

Course Contents

Unit 9: Software Quality Assurance (3 Hrs.)

Introduction to Software Quality, Software Quality Assurance, Software Reviews

Unit 10: Configuration Management (2 Hrs.)

Introduction to Version Management, Change Management and Release Management

Course Contents

Laboratory Works:

- Student should use project management tools focusing on resource management, project scheduling and people management.
- They should be use CASE tools for drawing UML diagrams.
- They should use tools for realizing agile development, version control and change control respectively.
- They should be able to develop test cases and use tools to demonstrate different types of testing.

References Books:

1. Software Engineering, 10th Edition, Ian Sommerville, Pearson Education 2016
2. Software Engineering: A Practitioner's Approach, 8th Edition, Roger S. Pressman and Bruce R. Maxim, McGraw-Hill Education 2015
3. C. Ghezzi, M. Jazayeri and D. Mandrioli, Fundamentals of Software Engineering Prentice Hall of India, Ltd.
4. G. Booch, J. Rambaugh, J. Jacobson, The unified Modeling Language – User Guide Addison - Wesley

Let's Start : Unit-1

Unit-1: Introduction

1. Professional Software Development
   - 1.1 Software engineering
     - What is software?
     - What are the attributes of good software?
     - What is software engineering?
     - What are the fundamental software engineering activities?
     - What is the difference between software engineering and computer science?
     - What is the difference between software engineering and system engineering?
     - What are the key challenges facing software engineering?
     - What are the costs of software engineering?
     - What are the best software engineering techniques and methods?
     - What differences has the Internet made to software engineering?
   - 1.2 Software engineering diversity
   - 1.3 Internet software engineering
2. Software Engineering Ethics
3. Case Studies

Unit-1: Introduction

Importance of Software Engineering:

- Virtually all countries now run depend on complex computer-based system.
- National infrastructures and utilities rely on computer-based systems that include a computer and controlling software.
- Industrial manufacturing and distribution is completely computerised, as is the financial system.
- Therefore , producing and maintaining software cost–effectively is essential for the functioning of national and international economies.
- Software Engineering is an engineering discipline whose focus is the cost-effective development of high-quality systems.
- As the evolution of new technologies in computer, communications and complex graphical user interfaces, a complex software is also require.
- So, new techniques and methods are required to control the complexity inherent in large software systems.

Unit-1: Introduction

Introduction:

- Software systems are abstract and intangible. They are not constrained by the properties of materials, nor are they governed by physical laws or by manufacturing processes.
- There are many different types of software system, ranging from simple embedded systems to complex, worldwide information systems.
- There are no universal notations, methods, or techniques for software engineering because different types of software require different approaches. Developing an organizational information system is completely different from developing a controller for a scientific instrument.
- All of these applications need software engineering; they do not all need the same software engineering methods and techniques.

Unit-1: Introduction

Reasons of software failures

Unit-1: Introduction

Software engineering is criticized as inadequate for modern software development. However, many of these so-called software failures are a consequence of two factors:

1. Increasing system complexity: Systems have to be built and delivered more quickly; larger, even more complex systems are required; and systems have to have new capabilities that were previously thought to be impossible. New software engineering techniques have to be developed to meet new the challenges of delivering more complex software.
2. Failure to use software engineering methods: Many companies do not use software engineering methods in their everyday work. Consequently, their software is often more expensive and less reliable than it should be. We need better software engineering education and training to address this problem.

Unit-1: Introduction

1. Professional Software Development:

- Lots of people/individual write programs. However, most software development is a professional activity in which software is developed for business purposes, for inclusion in other devices, or as software products such as information systems and computer-aided design systems.
- The key distinctions between individual and professional are that professional software is intended for use by someone apart from its developer and that teams rather than individuals usually develop the software. It is maintained and changed throughout its life.
- Software engineering is intended to support professional software development rather than individual programming. It includes techniques that support program specification, design, and evolution, none of which are normally relevant for personal software development.

Unit-1: Introduction

1. Professional Software Development:

- A professionally developed software system is often more than a single program. A system may consist of several separate programs and configuration files that are used to set up these programs. It may include system documentation, which describes the structure of the system, user documentation, which explains how to use the system, and websites for users to download recent product information. This is one of the important differences between professional and amateur(unprofessional) software development.

Unit-1: Introduction

What is software?

Software is a computer programs and associated documentation such as requirements, design models and user manuals. Software products may be developed for a particular customer or may be developed for a general market.

Software engineers are concerned with developing software products that can be sold to a customer. There are two kinds of software product:

1. Generic products - developed to be sold to a range of different customers e.g. PC software such as Excel or Word.
2. Customized (or bespoke) software - developed for a single customer according to their specification. A software contractor designs and implements the software especially for that customer. Examples are control systems for electronic devices, systems written to support a particular business process, and air traffic control systems.

New software can be created by developing new programs, configuring generic software systems or reusing existing software.

Unit-1: Introduction

What is software?

- Computer programs and associated documentation. Software products may be developed for a particular customer or may be developed for a general market.

What are the attributes of good software?

Good software should deliver the required functionality and performance to the user and should be maintainable, dependable and usable.

Unit-1: Introduction

What are the attributes of good software?

The software should deliver the required functionality and performance to the user and should be maintainable, dependable and acceptable.

1. **Maintainability**

   Software must evolve to meet changing needs of customers, business; Maintainability means "The ease with which a software system or component can be modified to correct faults, improve performance or other attributes, or adapt to a changed environment."

2. **Dependability and security**

   Software must be trustworthy; (i.e. Reliability, Security and Safety)

3. **Efficiency**

   Software should not make wasteful use of system resources (Processing time, memory utilization etc.);

4. **Acceptability**

   Software must accepted by the users for which it was designed. This means it must be understandable, usable and compatible with other systems. (i.e. User Interface and adequate documentation).

Unit-1: Introduction

What is Engineering?

- Engineering is the field that deals with the application of science, mathematics and other types of data to design and build up products and services that benefit the human livelihood.
- Engineering is broken down into many sub disciplines on the behalf of specialization in particular domains using various technologies.
- Software Engineering and Systems Engineering are two sub specialized disciplines that are not purely interconnected disciplines but they are intimately intertwined (linked).
- Although these two disciplines are interconnected to each other but at some phases they creates marginal difference.

Unit-1: Introduction

What is software engineering?

- Software engineering is not just concerned with the technical processes of software development. It also includes activities such as software project management and the development of tools, methods, and theories to support software development.
- In general, software engineers adopt a systematic and organized approach to their work, as this is often the most effective way to produce high-quality software.

Unit-1: Introduction

What is software engineering?

- Software engineering is an engineering discipline that is concerned with all aspects of software production.
- Software engineers should adopt a systematic and organised approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available.
- Software engineering concern about the designing and developments engineering specified software of the highest quality and should have broad information systems experience.
- Software engineering focus on cost effective development of high-quality software system.
- Software is an abstract and intangible.
- So, for cost effective & high-quality software system development require new techniques and methods to control the complexity inherent in large software systems.

Unit-1: Introduction

What is software engineering?

Software engineering is important for two reasons:

1. More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
2. It is usually cheaper, in the long run, to use software engineering methods and techniques for professional software systems rather than just write programs as a personal programming project. Failure to use software engineering method leads to higher costs for testing, quality assurance, and long-term maintenance.

Unit-1: Introduction

What is software process? (Generic software development process)

The systematic approach that is used in software engineering is called a software process. A software process is a sequence of activities that leads to the production of a software product. Four fundamental activities are common to all software processes.

1. **Software specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
2. **Software development**, where the software is designed and programmed.
3. **Software validation**, where the software is checked to ensure that it is what the customer requires.
4. **Software evolution**, where the software is modified to reflect changing customer and market requirements.

Unit-1: Introduction

What is system engineering?

- System engineering concerns on the overall management of engineering projects throughout their life cycle.
- System engineering highly concerns on the hardware part of the project. It deals with logistics, team coordination, automatic machinery control, work processes and similar tools.

Work of System Engineering?

- Operational concept development: This phase prepare the document of overall project development plan that describing the characteristics of a proposed system from the viewpoint of an individuals who will use that system.
- System integration: It defines the process of bringing collectively the component subsystems into one system and ensuring that the subsystems function together as a system.

Unit-1: Introduction

Work of System Engineering?

- Software Testing: It is a method of assessing the functionality of a software program. There are many different types of software capable to test proper software. Broadly testing is divided into two categories dynamic testing and static testing.
- Quality Analysis: It engage the fully software development process that includes monitoring and improving the process, and making sure that all the standard procedures are followed and ensuring that the problems are found and dealt with it.
- Risk managements and control: Risk management and control is the method by which software engineer identify the risk, reduce the impact of risk, reduce the probability or likelihood of risk and risk monitoring as well.
- Project opportunities analysis: It detailed review of the prospects for a software product within a possible market.

Unit-1: Introduction

Software Engineering and Computer Science?

- Computer science is concerned with theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
- Computer science theories are still insufficient to act as a complete underpinning for software engineering (unlike e.g. physics and electrical engineering).

Unit-1: Introduction

Software Engineering and System Engineering?

- Software engineers focus solely on software components while system engineering deals with a substantial amount of physical component of computers.
- Software engineering highly focuses on implementing quality software while system engineers highly concern about the users and domains.
- Software engineering includes in computer science or computer based engineering background while system engineering may covers a broader education area includes Engineering, Mathematics and Computer science.
- Software Engineering deals with designing and developing software of the highest quality, while Systems Engineering is the sub discipline of engineering, which deals with the overall management of engineering projects during their life cycle.
- Software engineering techniques such as use-case modelling and configuration management are being used in the systems engineering process.

Unit-1: Introduction

Software Engineering and System Engineering?

Note: System engineering overlaps with the concepts of industrial engineering, control engineering, organizational and project management and even software engineering. System Engineering is branded as an interdisciplinary engineering area. System Engineer may perform system designing, developing requirements, verifying requirements, system testing and other engineering studies.

So,

- System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering.
- Software engineering is part of this process concerned with developing the software infrastructure, control, applications and databases in the system.
- System engineers are involved in system specification, architectural design, integration and deployment.

Unit-1: Introduction

Difference between Software Engineering and Other Engineering?

- Engineering means solving problems of a real or potential customer which require design.
- Design follow a well-defined process to apply best practices from the particular engineering field to economically achieve a technological solution, accounting for constraints such as time-to-deliver, budget, and limitations arising from scientific laws, technology available, human factors (e.g. usability, accessibility to the disabled), human resources available, as well as safety, security, environmental, ethical and social concerns.
-

Unit-1: Introduction

What is software process?

Difference between Software Engineering and Other Engineering?

What are the key challenges facing software engineering?

Challenges and Cost of Software Engineering

Software Engineering Diversity:

Software Engineering Ethics

Unit-1: Introduction

Difference between Software Engineering and Other Engineering?

1. Technology available
2. Scientific laws
3. Human resources
4. Product type (Intangible)

1. Technology available: Software technology tends to more rapidly change, and be more complex, bug prone and under-specified than technology available to other engineers. There are formal methods and other processes such as test-driven development that can combat these concerns, but in software engineering we are far from the place where we can reliably and confidently use a component, knowing exactly how it will behave.

Unit-1: Introduction

2. Scientific laws: Much software is governed primarily by discrete mathematics (e.g. what is computable, and algorithmic complexity). However other engineering branches tend to be governed by continuous mathematics, as well as laws of physics (e.g. those of Newton, Ohm and so on.)

3. Human resources: Large numbers of people develop software without the level of training and rigorous licensing required in general of other engineers. There is no licensed software engineers.

- There are many 'computer scientists' are highly skilled; but anyone can call themselves a 'programmer' or 'software developer' without even an undergraduate degree in computer science or software engineering, and without a formal post-graduation process of building knowledge and skills under careful supervision.
- The result has been that a large amount of software is of relatively poor quality, including components that software engineers rely on to reuse.
- Many computer scientists and software engineers have put a lot of effort into making modern software more usable, but we are still some distance from ensuring all software developers know how to do that properly.

Unit-1: Introduction

4. Product is Intangible:

- You can see representations of it (code, models, etc.), but can't hold it. These days it is mostly transmitted over the Internet.
- Other engineered products need shipping or are built in place.
- Other engineers have to be concerned about the properties of the materials that will be used to construct their product, and there is a manufacturing process after the design is done.
- With software there are no 'materials', other than other intangible software, and there is a deployment step rather than a manufacturing step.
- We can change intangible software without having to throw away materials. Software is often changed without fully understanding the consequences, resulting in degradation of the design over time.
- Physical engineered products, on the other hand, degrade physically due to wear and chemical reactions.

Unit-1: Introduction

What are the key challenges facing software engineering?

There are many different types of software. There are no universal software engineering methods or techniques that may be used. However, there are four related issues that affect many different types of software:

- **Heterogeneity-** Systems are required to operate as distributed systems across networks that include different types of computer, mobile devices and tablets. Also have to integrate new software with older legacy systems written in different programming languages. The challenge here is to develop techniques for building dependable software that is flexible enough to cope with this heterogeneity.
- **Business and social change-** Businesses and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.
- **Security and trust-** We have to make sure that malicious users cannot successfully attack our software and that information security is maintained.

Unit-1: Introduction

- **Scale-** Software has to be developed across a very wide range of scales, from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.

To address these challenges, we will need new tools and techniques as well as innovative ways of combining and using existing software engineering methods.

Unit-1: Introduction

What are the best software engineering techniques and methods?

- While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed.

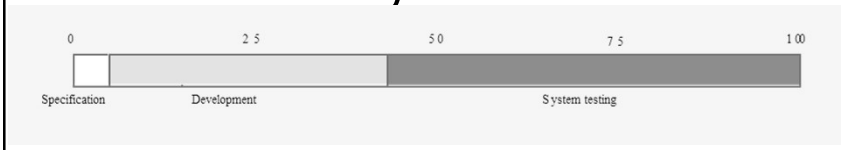
- There are no methods and techniques that are good for everything.

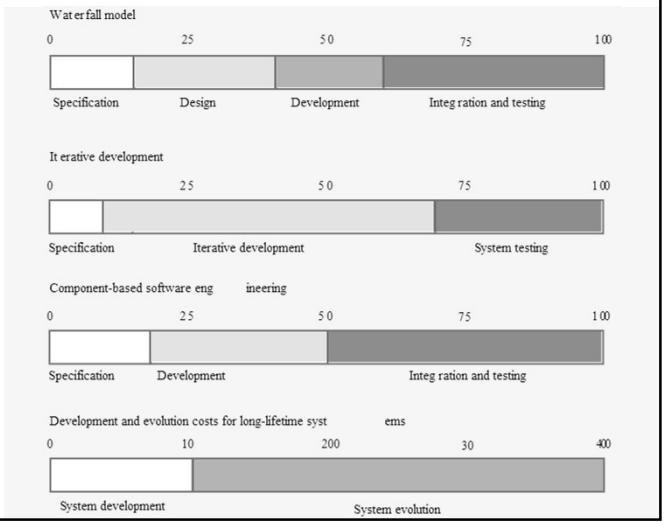
Unit-1: Introduction

Challenges and Cost of Software Engineering

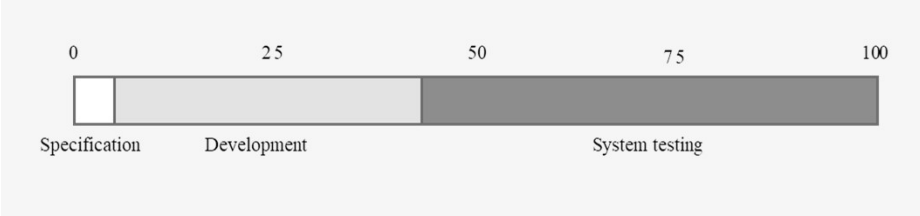
- Software engineering is concerned with cost-effective software development.
- Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.
- Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.
- Roughly 60% of costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
- Costs vary depending on the type of system being developed and the requirements of system attributes such as performance and system reliability.
- Distribution of costs depends on the development model that is used.

Activity cost distribution





Product development costs



Unit-1: Introduction

1.2 Software Engineering Diversity:

There are many different types of software system and there is no universal set of software techniques that is applicable to all of these.

The software engineering methods and tools used depend on the type of application being developed, the requirements of the customer and the background of the development team.

1. **Stand-alone applications-** These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
2. **Interactive transaction-based applications-** Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.

Unit-1: Introduction

3. Embedded control systems- These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

4. Batch processing systems- These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.

5. Entertainment systems- These are systems that are primarily for personal use and which are intended to entertain the user.

6. Systems for modeling and simulation- These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

7. Data collection systems- These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.

8. Systems of systems- These are systems that are composed of a number of other software systems.

Unit-1: Introduction

Software engineering fundamentals to all types of software systems

Each type of software system requires specialized software engineering techniques because the software has different characteristics. However, there are software engineering fundamentals that apply to all types of software systems:

Process -They should be developed using a managed and understood development process. The organization developing the software should plan the development process and have clear ideas of what will be produced and when it will be completed. Of course, the specific process that you should use depends on the type of software that you are developing.

Dependability- Dependability and performance are important for all types of system. Software should behave as expected, without failures, and should be available for use when it is required. It should be safe in its operation and, as far as possible, should be secure against external attack. The system should perform efficiently and should not waste resources.

Unit-1: Introduction

Requirements and management- Understanding and managing the software specification and requirements (what the software should do) are important. You have to know what different customers and users of the system expect from it, and you have to manage their expectations so that a useful system can be delivered within budget and to schedule.

Reuse- You should make effective use of existing resources. This means that, where appropriate, you should reuse software that has already been developed rather than write new software.

These fundamental notions of process, dependability, requirements, management, and reuse are important themes of software engineering. Different methods reflect them in different ways, but they underlie all professional software development.

These fundamentals are independent of the program language used for software development.

Unit-1: Introduction

1.3 Internet Software Engineering:

- THE INTERNET (a network of networks) has become the platform of choice for delivering services among people, business and others.
- The computing and application paradigms have been changed on the Internet because of the development of technologies like Web 2.0, social networking, mobile Internet, cloud computing, the Internet of Things, and big data.
- Internet Software engineering for Internet computing involves the architecting, development, deployment, management, and quality assurance of software supporting Internet-based systems. It also addresses global-development issues such as communication complexity, distributed control, governance policies, and cultural differences.

Unit-1: Introduction

2. Software Engineering Ethics

Professional and ethical responsibility:

- Software engineering involves wider responsibilities than simply the application of technical skills.
- Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- Ethical behaviour is more than simply upholding the law.

Unit-1: Introduction

Issues of professional responsibility:

Confidentiality- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

Competence- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is out with their competence.

Intellectual property rights- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

Computer misuse- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

Unit-1: Introduction

What are professional practices?

- The term 'professional practice' refers to the conduct and work of someone from a particular profession. Professional bodies may set standards of ethics, performance, competence, insurance, training and so on that must be met to remain within the profession.
- Software engineering has evolved into a respected, worldwide profession. As professionals, software engineers should abide by a code of ethics that guides the work that they do and the products that they produce. An ACM/IEEE-CS Joint Task Force has produced a Software Engineering Code of Ethics and Professional Practices (Version 5.1). The code [ACM98] states:
- Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Professional Practices

1. **PUBLIC**—Software engineers shall act consistently with the public interest.
2. **CLIENT AND EMPLOYER**—Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. **PRODUCT**—Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. **JUDGMENT**—Software engineers shall maintain integrity and independence in their professional judgment.
5. **MANAGEMENT**—Software engineering managers and leaders shall subscribe to and promote an ethical(moral) approach to the management of software development and maintenance.
6. **PROFESSION**—Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. **COLLEAGUES**—Software engineers shall be fair to and supportive of their colleagues.
8. **SELF**—Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.

Professional Practices

A software engineer should work in the public interest. On a personal level, a software engineer should abide by the following rules:

- Never steal data for personal gain.
- Never distribute or sell proprietary information obtained as part of your work on a software project.
- Never maliciously destroy or modify another person's programs, files, or data.
- Never violate the privacy of an individual, a group, or an organization.
- Never hack into a system for sport or profit.
- Never create or promulgate a computer virus or worm.
- Never use computing technology to facilitate discrimination or harassment.

In the context of software engineering, a process is not a rigid prescription for how to build computer software. Rather, it is an adaptable approach that enables the people doing the work (the software team) to pick and choose the appropriate set of work actions and tasks. The intent is always to deliver software in a timely manner and with sufficient quality to satisfy those who have sponsored its creation and those who will use it.

Unit-1: Introduction

3. Case Studies

To illustrate software engineering concepts, case studies of four different types of system are demonstrate as an example for clarity. Software engineering practice depends on the **type of systems** being produced.

Therefore have given an appropriate example when discussing concepts such as safety and dependability, system modeling, reuse, etc.

The system types that use as case studies are:

1. An embedded system: This is a system where the software controls some hardware device and is embedded in that device. Issues in embedded systems typically include physical size, responsiveness, and power management, etc. The example of an embedded system that use is a software system to control an **insulin pump** for people who have diabetes.

Unit-1: Introduction

3. Case Studies

The system types that use as case studies are:

2. An information system: The primary purpose of this type of system is to manage and provide access to a database of information. Issues in information systems include security, usability, privacy, and maintaining data integrity. The example of an information system used is a **medical records system**.

3. A sensor-based data collection system: This is a system whose primary purposes are to collect data from a set of sensors and to process that data in some way. The key requirements of such systems are reliability, even in hostile environmental conditions, and maintainability. The example of a data collection system that I use is a **wilderness weather station**.

4. A support environment: This is an integrated collection of software tools that are used to support some kind of activity. Here describe an example of a **digital learning environment** that is used to support students' learning in schools.

Unit-1: Introduction – Q&A

Unit-1: Introduction – Q&A

Unit-1: Introduction – Q&A

1. **Software failures are a consequence of two factors**
   a. Increasing system complexity:
   b. Failure to use software engineering methods:
2. **What are the attributes of good software?**
   a. Maintainability
   b. Dependability and security
   c. Efficiency
   d. Acceptability
3. **Software engineering is important for two reasons:**
   a. to produce reliable and trustworthy systems economically and quickly.
   b. leads to higher costs for testing, quality assurance, and long-term maintenance
4. **Four fundamental activities are common to all software processes.**
   a. Software specification
   b. Software development
   c. Software validation
   d. Software evolution
5. **Work of System Engineering?**
   a. Operational concept development:
   b. System integration:
   c. Software Testing
   d. Quality Analysis
   e. Risk managements and control
   f. Project opportunities analysis

Unit-1: Introduction

6. **Difference between Software Engineering and Other Engineering?**
   a. Technology available
   b. Scientific laws
   c. Human resources
   d. Product type (Intangible)
7. **What are the key challenges facing software engineering?**
   a. Heterogeneity
   b. Business and social change
   c. Security and trust
   d. Scale
   e. ……..
8. **Many different types of software system (Software Engineering Diversity)**
   a. Stand-alone applications
   b. Interactive transaction-based applications
   c. Embedded control systems
   d. Batch processing systems
   e. Entertainment systems
   f. Systems for modeling and simulation
   g. Data collection systems
   h. Systems of systems

Unit-1: Introduction

9. **Software engineering fundamentals to all types of software systems (software engineering fundamentals that apply to all types of software systems)**
    a. Process
    b. Dependability
    c. Requirements and management
    d. Reuse
10. **Issues of professional responsibility**
    a. Confidentiality
    b. Competence
    c. Intellectual property rights
    d. Computer misuse
11. **Professional Practices**
    a. PUBLIC
    b. CLIENT AND EMPLOYER
    c. PRODUCT
    d. JUDGMENT
    e. MANAGEMENT
    f. PROFESSION
    g. COLLEAGUES
    h. SELF